

 [Home](#) [MariaDB Platform](#) [Server](#) [MaxScale](#) [Analytics](#) [Galera Cluster](#) [Connectors](#) [Tools](#) [MariaDB Cloud](#)[REFERENCE](#) > [SQL STATEMENTS](#) > [ACCOUNT MANAGEMENT](#)

GRANT

Complete privilege management guide for MariaDB. Complete GRANT syntax for database, table, and column permissions with roles with comprehensive examples and.

Syntax

```

/* 1. Granting Privileges */
GRANT
    priv_type [(column_list)]
        [, priv_type [(column_list)]] ...
ON [object_type] priv_level
TO account_or_role [, account_or_role] ...
[REQUIRE {NONE | tls_option [[AND] tls_option] ...}]
[WITH grant_option_list]

/* 2. Granting Proxy Access */
GRANT PROXY ON user_or_role
    TO account_or_role [, account_or_role] ...
[WITH GRANT OPTION]

/* 3. Granting Roles */
GRANT role [, role] ...
    TO account_or_role [, account_or_role] ...
[WITH ADMIN OPTION]

/* Variable Definitions */
account_or_role:
    username [authentication_option]
    | role
    | PUBLIC
    | CURRENT_USER [()]
    | CURRENT_ROLE [()]

authentication_option:
    IDENTIFIED BY 'password'
    | IDENTIFIED BY PASSWORD 'password_hash'
    | IDENTIFIED {VIA | WITH} authentication_rule [OR authentication_rule ...]

authentication_rule:
    authentication_plugin
    | authentication_plugin {USING | AS} 'authentication_string'
    | authentication_plugin {USING | AS} PASSWORD('password')

priv_type:
    ALL [PRIVILEGES]
    | ALTER | ALTER ROUTINE | BINLOG ADMIN | BINLOG MONITOR | BINLOG REPLAY
    | CONNECTION ADMIN | CREATE | CREATE ROUTINE | CREATE TABLESPACE
    | CREATE TEMPORARY TABLES | CREATE USER | CREATE VIEW
    | DELETE | DELETE HISTORY | DROP | EVENT | EXECUTE | FEDERATED ADMIN
    | FILE | GRANT OPTION | INDEX | INSERT | LOCK TABLES | PROCESS
    | READ ONLY ADMIN | RELOAD | REPLICATION CLIENT | REPLICATION MASTER ADMIN
    | REPLICATION SLAVE | REPLICATION SLAVE ADMIN | REFERENCES
    | SELECT | SET USER | SHOW CREATE ROUTINE | SHOW DATABASES | SHOW VIEW
    | SHUTDOWN | SLAVE MONITOR | SUPER | TRIGGER | UPDATE | USAGE

object_type:
    TABLE
    | FUNCTION
    | PROCEDURE
    | PACKAGE
    | PACKAGE BODY

priv_level:
    *
    | *.*
    | db_name.*
    | db_name.tbl_name
    | tbl_name
    | db_name.routine_name

```

```

grant_option_list:
    grant_option [grant_option] ...

grant_option:
    GRANT OPTION
    | resource_option

resource_option:
    MAX_QUERIES_PER_HOUR count
    | MAX_UPDATES_PER_HOUR count
    | MAX_CONNECTIONS_PER_HOUR count
    | MAX_USER_CONNECTIONS count
    | MAX_STATEMENT_TIME time

tls_option:
    SSL
    | X509
    | CIPHER 'cipher'
    | ISSUER 'issuer'
    | SUBJECT 'subject'

```

Description

The `GRANT` statement allows you to grant privileges or [roles](#) to accounts. To use `GRANT`, you must have the `GRANT OPTION` privilege, and you must have the privileges that you are granting.

Use the [REVOKE](#) statement to revoke privileges granted with the `GRANT` statement.

Use the [SHOW GRANTS](#) statement to determine what privileges an account has.

Account Names

For `GRANT` statements, account names are specified as the `username` argument in the same way as they are for [CREATE USER](#) statements. See [account names](#) from the `CREATE USER` page for details on how account names are specified.

Implicit Account Creation

The `GRANT` statement also allows you to implicitly create accounts in some cases.

If the account does not yet exist, then `GRANT` can implicitly create it. To implicitly create an account with `GRANT`, a user is required to have the same privileges that would be required to explicitly create the account with the `CREATE USER` statement.

If the `NO_AUTO_CREATE_USER` [SQL_MODE](#) is set, then accounts can only be created if authentication information is specified, or with a [CREATE USER](#) statement. If no authentication information is provided, `GRANT` will produce an error when the specified account does not exist, for example:

```
SHOW VARIABLES LIKE '%sql_mode%';
```

```
+-----+-----+
| Variable_name | Value                                     |
+-----+-----+
| sql_mode      | NO_AUTO_CREATE_USER,NO_ENGINE_SUBSTITUTION |
+-----+-----+
```

```
GRANT USAGE ON *.* TO 'user123'@'%' IDENTIFIED BY '';
```

```
ERROR 1133 (28000): Can't find any matching row in the user table
```

```
GRANT USAGE ON *.* TO 'user123'@'%'
  IDENTIFIED VIA PAM using 'mariadb' require ssl ;
Query OK, 0 rows affected (0.00 sec)
```

```
SELECT host, user FROM mysql.user WHERE user='user123' ;
```

```
+-----+-----+
| host | user   |
+-----+-----+
| %    | user123 |
+-----+-----+
```

Privilege Levels

Privileges can be set globally, for an entire database, for a table or routine, or for individual columns in a table. Certain privileges can only be set at certain levels.

Global privileges do not take effect immediately and are only applied to connections created after the `GRANT` statement was executed.

- [Global privileges priv_type](#) are granted using `*.*` for `priv_level`. Global privileges include privileges to administer the database and manage user accounts, as well as privileges for all tables, functions, and procedures. Global privileges are stored in [mysql.global_priv table](#).
- [Database privileges priv_type](#) are granted using `db_name.*` for `priv_level`, or using just `*` to use the [current database](#). Database privileges include privileges to create tables and functions, as well as privileges for all tables, functions, and procedures in the database. Database privileges are stored in the [mysql.db table](#).
- [Table privileges priv_type](#) are granted using `db_name.tbl_name` for `priv_level`, or using just `tbl_name` to specify a table in the current database. The `TABLE` keyword is optional. Table privileges include the ability to select and change data in the table. Certain table privileges can be granted for individual columns.
- [Column privileges priv_type](#) are granted by specifying a table for `priv_level` and providing a column list after the privilege type. They allow you to control exactly which columns in a table users can select and change.
- [Function privileges priv_type](#) are granted using `FUNCTION db_name.routine_name` for `priv_level`, or using just `FUNCTION routine_name` to specify a function in the current database.

- [Procedure privileges `priv\_type`](#) are granted using `PROCEDURE db_name.routine_name` for `priv_level`, or using just `PROCEDURE routine_name` to specify a procedure in the current database.

The `USAGE` Privilege

The `USAGE` privilege grants no real privileges. The [SHOW GRANTS](#) statement will show a global `USAGE` privilege for a newly-created user. You can use `USAGE` with the `GRANT` statement to change options like `GRANT OPTION` and `MAX_USER_CONNECTIONS` without changing any account privileges.

The `ALL PRIVILEGES` Privilege

The `ALL PRIVILEGES` privilege grants all available privileges. Granting all privileges only affects the given privilege level. For example, granting all privileges on a table does not grant any privileges on the database or globally.

Using `ALL PRIVILEGES` does not grant the special `GRANT OPTION` privilege.

You can use `ALL` instead of `ALL PRIVILEGES`.

The `GRANT OPTION` Privilege

Use the `WITH GRANT OPTION` clause to give users the ability to grant privileges to other users at the given privilege level. Users with the `GRANT OPTION` privilege can only grant privileges they have. They cannot grant privileges at a higher privilege level than they have the `GRANT OPTION` privilege.

The `GRANT OPTION` privilege cannot be set for individual columns. If you use `WITH GRANT OPTION` when specifying [column privileges](#), the `GRANT OPTION` privilege will be granted for the entire table.

Using the `WITH GRANT OPTION` clause is equivalent to listing `GRANT OPTION` as a privilege.

Global Privileges

The following table lists the privileges that can be granted globally. You can also grant all database, table, and function privileges globally. When granted globally, these privileges apply to all databases, tables, or functions, including those created later.

To set a global privilege, use `*.*` for *priv_level*.

BINLOG ADMIN

Enables administration of the [binary log](#), including the [PURGE BINARY LOGS](#) statement and setting the system variables:

- [binlog_annotate_row_events](#)
- [binlog_cache_size](#)
- [binlog_commit_wait_count](#)
- [binlog_commit_wait_usec](#)
- [binlog_direct_non_transactional_updates](#)
- [binlog_expire_logs_seconds](#)
- [binlog_file_cache_size](#)
- [binlog_format](#)
- [binlog_row_image](#)
- [binlog_row_metadata](#)
- [binlog_stmt_cache_size](#)
- [expire_logs_days](#)
- [log_bin_compress](#)
- [log_bin_compress_min_len](#)
- [log_bin_trust_function_creators](#)
- [max_binlog_cache_size](#)
- [max_binlog_size](#)
- [max_binlog_stmt_cache_size](#)
- [sql_log_bin](#) and
- [sync_binlog](#).

BINLOG MONITOR

New name for [REPLICATION CLIENT](#). `REPLICATION CLIENT` can still be used, though.

Permits running SHOW commands related to the [binary log](#), in particular the [SHOW BINLOG STATUS](#) and [SHOW BINARY LOGS](#) statements.

BINLOG REPLAY

Enables replaying the binary log with the [BINLOG](#) statement (generated by [mariadb-binlog](#)), executing [SET timestamp](#) when [secure_timestamp](#) is set to `replication`, and setting the session values of system variables usually included in BINLOG output, in particular:

- [gtid_domain_id](#)
- [gtid_seq_no](#)
- [pseudo_thread_id](#)
- [server_id](#).

CONNECTION ADMIN

Enables administering connection resource limit options. This includes ignoring the limits specified by [max_user_connections](#) and [max_password_errors](#), and allowing one extra connection over [max_connections](#)

The statements specified in [init_connect](#) are not executed, [killing connections and queries](#) owned by other users is permitted. The following connection-related system variables can be changed:

- [connect_timeout](#)
- [disconnect_on_expired_password](#)
- [extra_max_connections](#)
- [init_connect](#)
- [max_connections](#)
- [max_connect_errors](#)
- [max_password_errors](#)
- [proxy_protocol_networks](#)
- [secure_auth](#)
- [slow_launch_time](#)
- [thread_pool_exact_stats](#)
- [thread_pool_dedicated_listener](#)
- [thread_pool_idle_timeout](#)
- [thread_pool_max_threads](#)
- [thread_pool_min_threads](#)
- [thread_pool_oversubscribe](#)
- [thread_pool_prio_kickup_timer](#)
- [thread_pool_priority](#)
- [thread_pool_size](#), and
- [thread_pool_stall_limit](#).

CREATE USER

Create a user using the [CREATE USER](#) statement, or implicitly create a user with the `GRANT` statement.

FEDERATED ADMIN

Current < 10.5

Execute [CREATE SERVER](#), [ALTER SERVER](#), and [DROP SERVER](#) statements.

FILE

Read and write files on the server, using statements like [LOAD DATA INFILE](#) or functions like [LOAD_FILE\(\)](#). Also needed to create [CONNECT](#) outward tables. MariaDB server must have the permissions to access those files.

GRANT OPTION

Grant global privileges. You can only grant privileges that you have.

PROCESS

Show information about the active processes, for example via [SHOW PROCESSLIST](#) or [mariadb-admin processlist](#). If you have the PROCESS privilege, you can see all threads. Otherwise, you can see only your own threads (that is, threads associated with the MariaDB account that you are using).

READ_ONLY ADMIN

Current < 10.11 < 10.5

User ignores the [read_only](#) system variable, and can perform write operations even when the `read_only` option is active.

A user with that privilege can also change the (global) value of `read_only`.

The `READ_ONLY ADMIN` privilege has been removed from [SUPER](#). The benefit of this is that one can remove the `READ_ONLY ADMIN` privilege from all users and ensure that no one can make any changes on any non-temporary tables. This is useful on replicas when one wants to ensure that the replica is kept identical to the primary.

RELOAD

Execute [FLUSH](#) statements or equivalent [mariadb-admin](#) commands.

REPLICATION CLIENT

Current < 10.5

Execute [SHOW MASTER STATUS](#) and [SHOW BINARY LOGS](#) informative statements. Renamed to [BINLOG MONITOR](#) (but still supported as an alias for compatibility reasons).

Current < 10.6

Execute [SHOW MASTER STATUS](#) and [SHOW BINARY LOGS](#) informative statements. Using [BINLOG MONITOR](#) instead is still supported as an alias.

REPLICATION MASTER ADMIN

Current < 10.5

Permits administration of primary servers, including the [SHOW REPLICATION HOSTS](#) statement, and setting the [gtid_binlog_state](#), [gtid_domain_id](#), [master_verify_checksum](#) and [server_id](#) system variables.

REPLICATION MONITOR

Current Reasoning < 10.5.9

Permit [SHOW REPLICATION STATUS](#) and [SHOW RELAYLOG EVENTS](#).

See *Reasoning* tab as to why this was implemented.

REPLICATION REPLICA

Current < 10.5

Synonym for [REPLICATION SLAVE](#).

REPLICATION SLAVE

Current < 10.5

Accounts used by replica servers on the primary need this privilege. This is needed to get the updates made on the master. [REPLICATION REPLICATION](#) is an alias for `REPLICATION SLAVE`.

REPLICATION SLAVE ADMIN

Permits administering replica servers, including [START REPLICATION/SLAVE](#), [STOP REPLICATION/SLAVE](#), [CHANGE MASTER](#), [SHOW REPLICATION/SLAVE STATUS](#), [SHOW RELAYLOG EVENTS](#) statements, replaying the binary log with the [BINLOG](#) statement (generated by [mariadb-binlog](#)), and setting the system variables:

- [gtid_cleanup_batch_size](#)
- [gtid_ignore_duplicates](#)
- [gtid_pos_auto_engines](#)
- [gtid_slave_pos](#)
- [gtid_strict_mode](#)
- [init_slave](#)
- [read_binlog_speed_limit](#)
- [relay_log_purge](#)
- [relay_log_recovery](#)
- [replicate_do_db](#)
- [replicate_do_table](#)
- [replicate_events_marked_for_skip](#)
- [replicate_ignore_db](#)
- [replicate_ignore_table](#)
- [replicate_wild_do_table](#)
- [replicate_wild_ignore_table](#)
- [slave_compressed_protocol](#)
- [slave_ddl_exec_mode](#)
- [slave_domain_parallel_threads](#)
- [slave_exec_mode](#)
- [slave_max_allowed_packet](#)
- [slave_net_timeout](#)
- [slave_parallel_max_queued](#)
- [slave_parallel_mode](#)
- [slave_parallel_threads](#)
- [slave_parallel_workers](#)
- [slave_run_triggers_for_rbr](#)
- [slave_sql_verify_checksum](#)
- [slave_transaction_retry_interval](#)
- [slave_type_conversions](#)
- [sync_master_info](#)
- [sync_relay_log](#), and
- [sync_relay_log_info](#).

SET USER

Current < 10.5

Enables setting the `DEFINER` when creating [triggers](#), [views](#), [stored functions](#) and [stored procedures](#).

SHOW DATABASES

List all databases using the [SHOW DATABASES](#) statement. Without the `SHOW DATABASES` privilege, you can still issue the `SHOW DATABASES` statement, but it will only list databases containing tables on which you have privileges.

SHUTDOWN

Shut down the server using [SHUTDOWN](#) or the [mariadb-admin shutdown](#) command.

SUPER

Execute superuser statements: [CHANGE MASTER TO](#), [KILL](#) (users who do not have this privilege can only `KILL` their own threads), [PURGE LOGS](#), [SET global system variables](#), or the [mariadb-admin debug](#) command. Also, this permission allows the user to write data even if the [read_only](#) startup option is set, enable or disable logging, enable or disable replication on replica, specify a `DEFINER` for statements that support that clause, connect once reaching the `MAX_CONNECTIONS`. If a statement has been specified for the [init-connect mariadb](#) option, that command will not be executed when a user with `SUPER` privileges connects to the server.

The SUPER privilege has been split into multiple smaller privileges to allow for more fine-grained privileges ([MDEV-21743 ↗](#)). The privileges are:

- [SET USER](#)
- [FEDERATED ADMIN](#)
- [CONNECTION ADMIN](#)
- [REPLICATION SLAVE ADMIN](#)
- [BINLOG ADMIN](#)
- [BINLOG REPLAY](#)
- [REPLICA MONITOR](#)
- [BINLOG MONITOR](#)
- [REPLICATION MASTER ADMIN](#)
- [READ_ONLY ADMIN](#)

These grants are no longer a part of SUPER and need to be granted separately.

The [READ_ONLY ADMIN](#) privilege has been removed from `SUPER`. The benefit of this is that one can remove the READ_ONLY ADMIN privilege from all users and ensure that no one can make any changes on any non-temporary tables. This is useful on replicas when one wants to ensure that the replica is kept identical to the primary ([MDEV-29596 ↗](#)).

Database Privileges

The following table lists the privileges that can be granted at the database level. You can also grant all table and function privileges at the database level. Table and function privileges on a database apply to all tables or functions in that database, including those created later.

To set a privilege for a database, specify the database using `db_name.*` for *priv_level*, or just use `*` to specify the current database.

Privilege	Description
CREATE	Create a database using the CREATE DATABASE statement, when the privilege is granted for a database. You can grant the CREATE privilege on databases that do not yet exist. This also grants the CREATE privilege on all tables in the database.
CREATE ROUTINE	Create Stored Programs using the CREATE PROCEDURE and CREATE FUNCTION statements.
CREATE TEMPORARY TABLES	Create temporary tables with the CREATE TEMPORARY TABLE statement. This privilege enable writing and dropping those temporary tables
DROP	Drop a database using the DROP DATABASE statement, when the privilege is granted for a database. This also grants the DROP privilege on all tables in the database.
EVENT	Create, drop and alter EVENTS.
GRANT OPTION	Grant database privileges. You can only grant privileges that you have.
LOCK TABLES	Acquire explicit locks using the LOCK TABLES statement; you also need to have the SELECT privilege on a table, in order to lock it.
SHOW CREATE ROUTINE	Permit viewing the SHOW CREATE definition statement of a routine, for example SHOW CREATE FUNCTION , even if not the routine owner. From MariaDB 11.3.0 .

Table Privileges

Privilege	Description
ALTER	Change the structure of an existing table using the ALTER TABLE statement.
CREATE	Create a table using the CREATE TABLE statement. You can grant the CREATE privilege on tables that do not yet exist.
CREATE VIEW	Create a view using the CREATE VIEW statement.
DELETE	Remove rows from a table using the DELETE statement.
DELETE HISTORY	Remove historical rows from a table using the DELETE HISTORY statement. Displays as DELETE VERSIONING ROWS when running SHOW PRIVILEGES until MariaDB 10.5.2 (MDEV-20382 ↗) . If a user has the SUPER privilege but not this privilege, running mariadb-upgrade will grant this privilege as well.
DROP	Drop a table using the DROP TABLE statement or a view using the DROP VIEW statement. Also required to execute the TRUNCATE TABLE statement.
GRANT OPTION	Grant table privileges. You can only grant privileges that you have.
INDEX	Create an index on a table using the CREATE INDEX statement. Without the INDEX privilege, you can still create indexes when creating a table using the CREATE TABLE statement if the you have the CREATE privilege, and you can create indexes using the ALTER TABLE statement if you have the ALTER privilege.
INSERT	Add rows to a table using the INSERT statement. The INSERT privilege can also be set on individual columns; see Column Privileges below for details.
REFERENCES	Unused.
SELECT	Read data from a table using the SELECT statement. The SELECT privilege can also be set on individual columns; see Column Privileges below for details.
SHOW VIEW	Show the CREATE VIEW statement to create a view using the SHOW CREATE VIEW statement.
TRIGGER	Required to run the CREATE TRIGGER , DROP TRIGGER , and SHOW CREATE TRIGGER statements. When another user activates a trigger (running INSERT, UPDATE, or DELETE statements on the associated table), for the trigger to execute, the user that defined the trigger should have the TRIGGER privilege for the table. The user running the INSERT, UPDATE, or DELETE statements on the table is not required to have the TRIGGER privilege.
UPDATE	Update existing rows in a table using the UPDATE statement. UPDATE statements usually include a WHERE clause to update only certain rows. You must have SELECT privileges on the table or the appropriate columns for the WHERE clause. The UPDATE privilege can also be set on individual columns; see Column Privileges below for details.

Column Privileges

Some table privileges can be set for individual columns of a table. To use column privileges, specify the table explicitly and provide a list of column names after the privilege type. For example, the following statement would allow the user to read the names and positions of employees, but not other information from the same table, such as salaries.

```
GRANT SELECT (name, position) ON Employee TO 'jeffrey'@'localhost';
```

Privilege	Description
INSERT (column_list)	Add rows specifying values in columns using the INSERT statement. If you only have column-level <code>INSERT</code> privileges, you must specify the columns you are setting in the <code>INSERT</code> statement. All other columns will be set to their default values, or <code>NULL</code> .
REFERENCES (column_list)	Unused.
SELECT (column_list)	Read values in columns using the SELECT statement. You cannot access or query any columns for which you do not have <code>SELECT</code> privileges, including in <code>WHERE</code> , <code>ON</code> , <code>GROUP BY</code> , and <code>ORDER BY</code> clauses.
UPDATE (column_list)	Update values in columns of existing rows using the UPDATE statement. <code>UPDATE</code> statements usually include a <code>WHERE</code> clause to update only certain rows. You must have <code>SELECT</code> privileges on the table or the appropriate columns for the <code>WHERE</code> clause.

Function Privileges

Privilege	Description
ALTER ROUTINE	Change the characteristics of a stored function using the ALTER FUNCTION statement.
EXECUTE	Use a stored function. You need <code>SELECT</code> privileges for any tables or columns accessed by the function.
GRANT OPTION	Grant function privileges. You can only grant privileges that you have.

Procedure Privileges

Privilege	Description
ALTER ROUTINE	Change the characteristics of a stored procedure using the ALTER PROCEDURE statement.
EXECUTE	Execute a stored procedure using the CALL statement. The privilege to call a procedure may allow you to perform actions you wouldn't otherwise be able to do, such as insert rows into a table.
GRANT OPTION	Grant procedure privileges. You can only grant privileges that you have.

```
GRANT EXECUTE ON PROCEDURE mysql.create_db TO maintainer;
```

Package Privileges

Privilege	Description
ALTER ROUTINE	Change the characteristics of a stored package.
EXECUTE	Execute a stored package or package body.
GRANT OPTION	Grant package privileges. You can only grant privileges that you have.

Proxy Privileges

Privilege	Description
PROXY	Permits one user to be a proxy for another.

The `PROXY` privilege allows one user to proxy as another user, which means their privileges change to that of the proxy user, and the `CURRENT_USER()` function returns the user name of the proxy user.

The `PROXY` privilege only works with authentication plugins that support it. The default `mysql_native_password` authentication plugin does not support proxy users.

The `pam` authentication plugin is the only plugin included with MariaDB that currently supports proxy users. The `PROXY` privilege is commonly used with the `pam` authentication plugin to enable [user and group mapping with PAM](#).

For example, to grant the `PROXY` privilege to an [anonymous account](#) that authenticates with the `pam` authentication plugin, you could execute the following:

```
CREATE USER 'dba'@'%' IDENTIFIED BY 'strongpassword';
GRANT ALL PRIVILEGES ON *.* TO 'dba'@'%' ;

CREATE USER ''@'%' IDENTIFIED VIA pam USING 'mariadb';
GRANT PROXY ON 'dba'@'%' TO ''@'%';
```

A user account can only grant the `PROXY` privilege for a specific user account if the granter also has the `PROXY` privilege for that specific user account, and if that privilege is defined `WITH GRANT OPTION`. For example, the following example fails because the granter does not have the `PROXY` privilege for that specific user account at all:

```
SELECT USER(), CURRENT_USER();
```



```
+-----+
| USER()          | CURRENT_USER() |
+-----+
| alice@localhost | alice@localhost |
+-----+
```

SHOW GRANTS

```
+-----+
| Grants for alice@localhost
|
+-----+
| GRANT ALL PRIVILEGES ON *.* TO 'alice'@'localhost' IDENTIFIED BY PASSWORD
'|*2470C0C06DEE42FD1618BB99005ADCA2EC9D1E19' |
+-----+
```

```
GRANT PROXY ON 'dba'@'localhost' TO 'bob'@'localhost';
ERROR 1698 (28000): Access denied for user 'alice'@'localhost'
```

And the following example fails because the granter does have the `PROXY` privilege for that specific user account, but it is not defined `WITH GRANT OPTION` :

```
SELECT USER(), CURRENT_USER();
```

```
+-----+
| USER()          | CURRENT_USER() |
+-----+
| alice@localhost | alice@localhost |
+-----+
```

SHOW GRANTS;

```
+-----+
| Grants for alice@localhost
|
+-----+
| GRANT ALL PRIVILEGES ON *.* TO 'alice'@'localhost' IDENTIFIED BY PASSWORD
'|*2470C0C06DEE42FD1618BB99005ADCA2EC9D1E19' |
| GRANT PROXY ON 'dba'@'localhost' TO 'alice'@'localhost'
|
+-----+
```

```
GRANT PROXY ON 'dba'@'localhost' TO 'bob'@'localhost';
ERROR 1698 (28000): Access denied for user 'alice'@'localhost'
```

But the following example succeeds because the granter does have the `PROXY` privilege for that specific user account, and it is defined `WITH GRANT OPTION` :

```
SELECT USER(), CURRENT_USER();
```

```
+-----+-----+
| USER()          | CURRENT_USER() |
+-----+-----+
| alice@localhost | alice@localhost |
+-----+-----+
```

```
SHOW GRANTS;
```

```
+-----+
+-----+
| Grants for alice@localhost
|
+-----+
+-----+
| GRANT ALL PRIVILEGES ON *.* TO 'alice'@'localhost' IDENTIFIED BY PASSWORD
'|*2470C0C06DEE42FD1618BB99005ADCA2EC9D1E19' WITH GRANT OPTION |
| GRANT PROXY ON 'dba'@'localhost' TO 'alice'@'localhost' WITH GRANT OPTION
|
+-----+
+-----+
```

```
GRANT PROXY ON 'dba'@'localhost' TO 'bob'@'localhost';
```

A user account can grant the `PROXY` privilege for any other user account if the granter has the `PROXY` privilege for the `'@'%'` anonymous user account, like this:

```
GRANT PROXY ON ''@%' TO 'dba'@'localhost' WITH GRANT OPTION;
```

For example, the following example succeeds because the user can grant the `PROXY` privilege for any other user account:

```
SELECT USER(), CURRENT_USER();
```

```
+-----+-----+
| USER()          | CURRENT_USER() |
+-----+-----+
| alice@localhost | alice@localhost |
+-----+-----+
```

```
SHOW GRANTS;
```

```
+-----+
+-----+
| Grants for alice@localhost
|
+-----+
+-----+
| GRANT ALL PRIVILEGES ON *.* TO 'alice'@'localhost' IDENTIFIED BY PASSWORD
'|*2470C0C06DEE42FD1618BB99005ADCA2EC9D1E19' WITH GRANT OPTION |
| GRANT PROXY ON ''@%' TO 'alice'@'localhost' WITH GRANT OPTION
|
+-----+
+-----+
```

```
GRANT PROXY ON 'app1_dba'@'localhost' TO 'bob'@'localhost';
Query OK, 0 rows affected (0.004 sec)
```

```
GRANT PROXY ON 'app2_dba'@'localhost' TO 'carol'@'localhost';
Query OK, 0 rows affected (0.004 sec)
```

The default `root` user accounts created by [mariadb-install-db](#) have this privilege. For example:

```
GRANT ALL PRIVILEGES ON *.* TO 'root'@'localhost' WITH GRANT OPTION;
GRANT PROXY ON ''@%' TO 'root'@'localhost' WITH GRANT OPTION;
```

This allows the default `root` user accounts to grant the `PROXY` privilege for any other user account, and it also allows the default `root` user accounts to grant others the privilege to do the same.

Authentication Options

The authentication options for the `GRANT` statement are the same as those for the [CREATE USER](#) statement.

IDENTIFIED BY 'password'

The optional `IDENTIFIED BY` clause can be used to provide an account with a password. The password should be specified in plain text. It will be hashed by the [PASSWORD](#) function prior to being stored.

For example, if our password is `mariadb`, then we can create the user with:

```
GRANT USAGE ON *.* TO foo2@test IDENTIFIED BY 'mariadb';
```

If you do not specify a password with the `IDENTIFIED BY` clause, the user will be able to connect without a password. A blank password is not a wildcard to match any password. The user must connect without providing a password if no password is set.

If the user account already exists and if you provide the `IDENTIFIED BY` clause, then the user's password will be changed. You must have the privileges needed for the [SET PASSWORD](#) statement to change a user's password with `GRANT`.

The only [authentication plugins](#) that this clause supports are [mysql_native_password](#) and [mysql_old_password](#).

IDENTIFIED BY PASSWORD 'password_hash'

The optional `IDENTIFIED BY PASSWORD` clause can be used to provide an account with a password that has already been hashed. The password should be specified as a hash that was provided by the [PASSWORD](#) function. It will be stored as-is.

For example, if our password is `mariadb`, then we can find the hash with:

```
SELECT PASSWORD('mariadb');
```

```
+-----+
| PASSWORD('mariadb') |
+-----+
| *54958E764CE10E50764C2EECB71D01F08549980 |
+-----+
1 row in set (0.00 sec)
```

And then we can create a user with the hash:

```
GRANT USAGE ON *.* TO foo2@test IDENTIFIED BY
PASSWORD '*54958E764CE10E50764C2EECB71D01F08549980';
```

If you do not specify a password with the `IDENTIFIED BY` clause, the user will be able to connect without a password. A blank password is not a wildcard to match any password. The user must connect without providing a password if no password is set.

If the user account already exists and if you provide the `IDENTIFIED BY` clause, then the user's password will be changed. You must have the privileges needed for the [SET PASSWORD](#) statement to change a user's password with `GRANT`.

The only [authentication plugins](#) that this clause supports are [mysql_native_password](#) and [mysql_old_password](#).

IDENTIFIED {VIA|WITH} authentication_plugin

The optional `IDENTIFIED VIA authentication_plugin` allows you to specify that the account should be authenticated by a specific [authentication plugin](#). The plugin name must be an active authentication plugin as per [SHOW PLUGINS](#). If it doesn't show up in that output, then you will need to install it with [INSTALL PLUGIN](#) or [INSTALL SONAME](#).

For example, this could be used with the [PAM authentication plugin](#):

```
GRANT USAGE ON *.* TO foo2@test IDENTIFIED VIA pam;
```

Some authentication plugins allow additional arguments to be specified after a `USING` or `AS` keyword. For example, the [PAM authentication plugin](#) accepts a [service name](#):

```
GRANT USAGE ON *.* TO foo2@test IDENTIFIED VIA pam USING 'mariadb';
```

The exact meaning of the additional argument would depend on the specific authentication plugin.

The `USING` or `AS` keyword can also be used to provide a plain-text password to a plugin if it's provided as an argument to the `PASSWORD()` function. This is only valid for [authentication plugins](#) that have implemented a hook for the `PASSWORD()` function. For example, the [ed25519](#) authentication plugin supports this:

```
CREATE USER safe@%' IDENTIFIED VIA ed25519
  USING PASSWORD('secret');
```

One can specify many authentication plugins, they all work as alternative ways of authenticating a user:

```
CREATE USER safe@%' IDENTIFIED VIA ed25519
  USING PASSWORD('secret') OR unix_socket;
```

By default, when you create a user without specifying an authentication plugin, MariaDB uses the [mysql_native_password](#) plugin.

Resource Limit Options

It is possible to set per-account limits for certain server resources. The following table shows the values that can be set per account:

Limit Type	Description
<code>MAX_QUERIES_PER_HOUR</code>	Number of statements that the account can issue per hour (including updates)
<code>MAX_UPDATES_PER_HOUR</code>	Number of updates (not queries) that the account can issue per hour
<code>MAX_CONNECTIONS_PER_HOUR</code>	Number of connections that the account can start per hour
<code>MAX_USER_CONNECTIONS</code>	Number of simultaneous connections that can be accepted from the same account; if it is 0, <code>max_connections</code> will be used instead; if <code>max_connections</code> is 0, there is no limit for this account's simultaneous connections.
<code>MAX_STATEMENT_TIME</code>	Timeout, in seconds, for statements executed by the user. See also Aborting Statements that Exceed a Certain Time to Execute .

If any of these limits are set to `0`, then there is no limit for that resource for that user.

To set resource limits for an account, if you do not want to change that account's privileges, you can issue a `GRANT` statement with the `USAGE` privilege, which has no meaning. The statement can name some or all limit types, in any order.

Here is an example showing how to set resource limits:

```
GRANT USAGE ON *.* TO 'someone'@'localhost' WITH
MAX_USER_CONNECTIONS 0
MAX_QUERIES_PER_HOUR 200;
```

The resources are tracked per account, which means `'user'@'server'` ; not per user name or per connection.

The count can be reset for all users using [FLUSH USER_RESOURCES](#), [FLUSH PRIVILEGES](#) or [mariadb-admin reload](#).

Current < 10.5

Users with the `CONNECTION ADMIN` privilege or the `SUPER` privilege are not restricted by `max_user_connections` or `max_password_errors` , and they are allowed one additional connection when `max_connections` is reached.

Per account resource limits are stored in the [user](#) table, in the [mysql](#) database. Columns used for resources limits are named `max_questions` , `max_updates` , `max_connections` (for `MAX_CONNECTIONS_PER_HOUR`), and `max_user_connections` (for `MAX_USER_CONNECTIONS`).

TLS Options

By default, MariaDB transmits data between the server and clients without encrypting it. This is generally acceptable when the server and client run on the same host or in networks where security is guaranteed through other means. However, in cases where the server and client exist on separate networks or they are in a high-risk network, the lack of encryption does introduce security concerns as a malicious actor could potentially eavesdrop on the traffic as it is sent over the network between them.

To mitigate this concern, MariaDB allows you to encrypt data in transit between the server and clients using the Transport Layer Security (TLS) protocol. TLS was formerly known as Secure Socket Layer (SSL), but strictly speaking the SSL protocol is a predecessor to TLS and, that version of the protocol is now considered insecure. The documentation still uses the term SSL often and for compatibility reasons TLS-related server system and status variables still use the prefix `ssl_`, but internally, MariaDB only supports its secure successors.

See [Secure Connections Overview](#) for more information about how to determine whether your MariaDB server has TLS support.

You can set certain TLS-related restrictions for specific user accounts. For instance, you might use this with user accounts that require access to sensitive data while sending it across networks that you do not control. These restrictions can be enabled for a user account with the [CREATE USER](#), [ALTER USER](#), or [GRANT](#) statements. The following options are available:

Option	Description
REQUIRE NONE	TLS is not required for this account, but can still be used.
REQUIRE SSL	The account must use TLS, but no valid X509 certificate is required. This option cannot be combined with other TLS options.
REQUIRE X509	The account must use TLS and must have a valid X509 certificate. This option implies REQUIRE SSL. This option cannot be combined with other TLS options.
REQUIRE ISSUER 'issuer'	The account must use TLS and must have a valid X509 certificate. Also, the Certificate Authority must be the one specified via the string issuer. This option implies REQUIRE X509. This option can be combined with the SUBJECT, and CIPHER options in any order.
REQUIRE SUBJECT 'subject'	The account must use TLS and must have a valid X509 certificate. Also, the certificate's Subject must be the one specified via the string subject. This option implies REQUIRE X509. This option can be combined with the ISSUER, and CIPHER options in any order.
REQUIRE CIPHER 'cipher'	The account must use TLS, but no valid X509 certificate is required. Also, the encryption used for the connection must use a specific cipher method specified in the string cipher. This option implies REQUIRE SSL. This option can be combined with the ISSUER, and SUBJECT options in any order.

The `REQUIRE` keyword must be used only once for all specified options, and the `AND` keyword can be used to separate individual options, but it is not required.

For example, you can create a user account that requires these TLS options with the following:

```
GRANT USAGE ON *.* TO 'alice'@%'
  REQUIRE SUBJECT '/CN=alice/O=My Dom, Inc./C=US/ST=Oregon/L=Portland'
  AND ISSUER '/C=FI/ST=Somewhere/L=City/ O=Some Company/CN=Peter Parker/
emailAddress=p.parker@marvel.com'
  AND CIPHER 'SHA-DES-CBC3-EDH-RSA';
```

If any of these options are set for a specific user account, then any client who tries to connect with that user account will have to be configured to connect with TLS.

See [Securing Connections for Client and Server](#) for information on how to enable TLS on the client and server.

Roles

```
GRANT role TO grantee [, grantee ... ]
[ WITH ADMIN OPTION ]

grantee:
  rolename
  username [authentication_option]
```

The `GRANT` statement is also used to grant the use of a [role](#) to one or more users or other roles. In order to be able to grant a role, the grantor doing so must have permission to do so (see `WITH ADMIN` in the [CREATE ROLE](#) article).

Specifying the `WITH ADMIN OPTION` permits the grantee to in turn grant the role to another.

For example, the following commands show how to grant the same role to a couple different users.

```
GRANT journalist TO hulda;  
  
GRANT journalist TO berengar WITH ADMIN OPTION;
```

If a user has been granted a role, they do not automatically obtain all permissions associated with that role. These permissions are only in use when the user activates the role with the [SET ROLE](#) statement.

 Be careful to avoid conflicting role and user names. In case of a conflict, the role name takes precedence, as shown in the following example. The `GRANT` statement assigns privileges to the role, not to the user:

```
CREATE USER alice IDENTIFIED BY 'password';  
CREATE ROLE alice;  
GRANT select, insert on db.* TO alice;
```

TO PUBLIC

Syntax

```
GRANT <privilege> ON <db_name>.<object> TO PUBLIC;  
REVOKE <privilege> ON <db_name>.<object> FROM PUBLIC;
```

`GRANT ... TO PUBLIC` grants privileges to all users with access to the server. The privileges also apply to users created after the privileges are granted. This can be useful when you only want to state once that all users need to have a certain set of privileges. When running [SHOW GRANTS](#), a user also sees all privileges inherited from `PUBLIC`. [SHOW GRANTS FOR PUBLIC](#) only shows `TO PUBLIC` grants.

Example

The following example shows the difference between granting privileges to particular users and granting privileges to `PUBLIC`.

```
-- ... (connect as user root) ...  
MariaDB [(none)]> CREATE USER developer;  
MariaDB [(none)]> CREATE DATABASE dev_db;  
MariaDB [(none)]> GRANT ALL ON dev_db.* TO PUBLIC;  
MariaDB [(none)]> GRANT ALL ON mysql.* TO developer;  
-- ... (connect as user developer) ...  
MariaDB [(none)]> SHOW GRANTS;  
+-----+  
| Grants for developer@% |  
+-----+  
| GRANT USAGE ON . TO developer@% |  
| GRANT ALL PRIVILEGES ON mysql.* TO developer@% |  
| GRANT ALL PRIVILEGES ON dev_db.* TO PUBLIC |  
+-----+  
MariaDB [(none)]> SHOW GRANTS FOR PUBLIC;  
+-----+  
| Grants for PUBLIC |  
+-----+  
| GRANT ALL PRIVILEGES ON `dev_db`.* TO `PUBLIC` |  
+-----+
```

For more details, and information on the background of this feature, refer to this [blog post](#).

Grant Examples

Granting Root-like Privileges

You can create a user that has privileges similar to the default `root` accounts by executing the following:

```
CREATE USER 'alexander'@'localhost';  
GRANT ALL PRIVILEGES ON *.* TO 'alexander'@'localhost' WITH GRANT OPTION;
```

See Also

- [Troubleshooting Connection Issues](#)
- [Authentication from MariaDB 10.4](#)
- [--skip-grant-tables](#) allows you to start MariaDB without `GRANT` . This is useful if you lost your root password.
- [CREATE USER](#)
- [ALTER USER](#)
- [DROP USER](#)
- [SET PASSWORD](#)
- [SHOW CREATE USER](#)
- [mysql.global_priv](#) table
- [mysql.user](#) table
- [Password Validation Plugins](#) - permits the setting of basic criteria for passwords
- [Authentication Plugins](#) - allow various authentication methods to be used, and new ones to be developed.

This page is licensed: GPLv2, originally from [fill_help_tables.sql](#) ↗

Subscribe to our newsletter!



Previous
[DROP USER](#)

Next
[RENAME USER](#)



Last updated 4 months ago

Was this helpful?



Products	Support	Resources	Company
Enterprise Platform	Customer Login	MariaDB Blog	About MariaDB
Community Server	Technical Support	Webinars	Newsroom
Download MariaDB	Remote DBA	Customer Stories	Leadership
Pricing	Professional Services	MariaDB Events	MariaDB Careers
		Documentation	Legal
		Developer Hub	Privacy Policy